WHAT THIS DOCUMENT FIXES

On 03-04 May 2026 Charlie and Jordan ran a real bench test. Wiring was good, the rack was patched and accepting commands, motion was real and at times "butter smooth". But the rack kept flickering between EAC=ACTIVE, EAC=AVAILABLE, and occasionally EAC=INHIBITED, with the error code most often reading **HIGH_ANGLE_RATE_REQ**, sometimes HIGH_ANGLE_REQ, and once OUT_OF_RANGE. The flicker itself caused visible twitching of the wheel during what should have been smooth ramps.

This guide gives you a structured path: diagnose what kind of flicker you have, fix it, then verify. It assumes you have already completed PINOUT_VERIFICATION, INSTRUCTION_MANUAL, and FLASH_AND_TROUBLESHOOTING (the rack is patched and accepting 0x488 commands).

BACKGROUND: WHY EAC FLICKERS

The rack runs an internal state machine for steering control. When we send 0x488 DAS_steeringControl with controlType=1, the rack transitions:

INHIBITED > AVAILABLE > ACTIVE

Each transition requires the rack to satisfy its own internal sanity checks: angle within limit, rate of change within limit, no driver torque on the wheel, EPB/GTW handshake messages present, valid checksum and incrementing counter on every frame. **If any check fails on any single frame**, the rack drops back one state. The next valid frame moves it forward again. Repeated quick failures produce the flicker pattern.

The rack's eacErrorCode field tells you WHICH check failed most recently. Watching that field over time is your primary diagnostic tool.

EAC ERROR CODE REFERENCE (what each one means for our build)

Code	Name	Meaning in our context	Fix
0	NONE	No error. EAC may still be in INHIBITED if not engaged.	Normal
1	MIN_SPEED	Rack thinks car is stationary; standstill safety. Common at 0 km/h.	Drive >3 mph OR enable BENCH_MODE
2	MAX_SPEED	Rack thinks car is too fast for steering. Should not happen at standstill.	Check 0x155 traffic; bad data?
3	HANDS_ON	Driver applied torque on wheel.	Let go of the wheel completely
4	OUT_OF_RANGE	Commanded angle outside rack-allowed window for current speed.	Stay below +/- 60 deg at standstill
5	OVER_TORQUE	Motor demanded too much current; usually loaded wheels at standstill.	Get wheels off the ground
6	HIGH_ANGLE_REQ	Commanded angle exceeds the rack's speed-scaled limit.	Lower commanded angle; max ~60 deg at 0 km/h
7	HIGH_ANGLE_RATE_REQ	**The big one Charlie saw.** Commanded angle changed faster than rack allows.	Slow the rate. See pages 2-3.
8	HIGH_TORQUE_REQ	Rack would have to apply more torque than allowed.	Wheels-off the ground OR slow rate
9	BLEND_REQ	Driver and EPS commanded different things; rack faulting in handoff.	Don't fight the wheel during engage
10	TIMEOUT	Rack did not receive 0x488 frequently enough.	Check our 50 Hz; check bus errors
11	ECU_FAULT	Internal fault.	Power-cycle the rack
12	BUS_FAULT	CAN bus electrical issue.	Check H/L wiring; termination
13	INVALID_REQ	Bad checksum or counter on a 0x488 frame.	Should not happen with our code
14	EPB_INHIBIT	EPB module told rack to disengage.	Brake pedal pressed?
15	SNA	Signal not available; rack not initialized.	Wait or power-cycle

STEP 1 -- IDENTIFY YOUR FLICKER PATTERN

Pull main from GitHub before this step. The latest GUI logs every EAC transition with the current error code in real time, so you can read flicker patterns directly from the event log instead of taking screenshots.

```
git pull origin main
```

Connect, ENGAGE at 0 deg, then slide to +5. Watch the event log for transition messages. Match the pattern to one of these four:

Pattern	Most common error	Likely cause	Path to take
ACTIVE -> AVAILABLE -> ACTIVE every ~100 ms during ramp	HIGH_ANGLE_RATE_REQ	Our commanded angle is moving too fast. The rack's rate-of-change check rejects each frame.	**Page 3, FIX A or FIX B.**
ACTIVE briefly, then INHIBITED, stays there	HIGH_ANGLE_REQ or OUT_OF_RANGE	Commanded angle exceeds the speed-scaled limit. Most common at >+/-60 deg standstill.	**Page 3, FIX C.**
AVAILABLE -> INHIBITED -> AVAILABLE before you can ENGAGE	MIN_SPEED or various	Rack is gating on speed. Stock ESP says 0 km/h, rack waits for movement.	**Page 3, FIX D.**
Random twitching with no clear pattern	Mixed errors	Bus contention or electrical noise. Bus error count rising.	**Page 4, FIX E.**

STEP 2 -- COLLECT DATA BEFORE CHANGING ANYTHING

Run these checks while the GUI is engaged and you're seeing the flicker. Note the readings:

1. **Bus Errors counter**: should stay at 0. If it climbs, it is electrical and you need FIX E.
2. **0x370 RX counter**: should climb steadily, ~100 per second. If it stalls, the rack is not heard us.
3. **Measured Angle vs Commanded Angle**: in steady state they should agree to within 1-2 deg. Big divergence (> 5 deg) during ramp = rack is not following our command, it is rejecting frames.
4. **Slider response time**: time from slider drop to wheel arriving at target. Should be smooth and finish within ~1 sec for a 30 deg step.

WHAT THE THEORIES ARE BETTING ON

I have built two branches in the GitHub repo, each implementing a different fix theory. Pick one based on your flicker pattern, or A/B test both:

Branch	Theory	Best for	Trade-off
theory-A-conservative-rate	Rate is too aggressive at standstill. Drop MAX_RATE to 20 deg/s.	HIGH_ANGLE_RATE_REQ during ramps	Big-angle commands take 1.5 sec instead of 0.6 sec
theory-B-pure-filter	Rate-limiter velocity discontinuity is the culprit. Remove rate limiter, LPF only.	HIGH_ANGLE_RATE_REQ AND/OR jerky feel at end of ramps	Motion has organic ease-in/ease-out instead of constant speed

FIX A -- LOWER THE RATE LIMIT (Theory A branch)

Use this when HIGH_ANGLE_RATE_REQ is the dominant error and motion looks jerky in steps.

3A.1 -- Switch to the theory-A branch:

```
git fetch origin
git checkout theory-A-conservative-rate
```

3A.2 -- Rerun the GUI:

```
python tesla_steering_test.py
```

3A.3 -- Connect, engage at 0, slide to +5. Watch event log. If HIGH_ANGLE_RATE_REQ disappears, Theory A wins. If it still appears, try Fix B.

Tunable: open the script, find these lines near the top:

```
MAX_RATE_DEG_PER_SEC = 20.0 # was 50
TARGET_FILTER_TAU_S = 0.30 # was 0.15
```

If 20 deg/s still triggers HIGH_ANGLE_RATE_REQ, drop further to 10 or even 5. The rack at perfect standstill may want very slow rates.

FIX B -- REMOVE THE RATE LIMITER ENTIRELY (Theory B branch)

Use this when Fix A doesn't help, or when the wheel motion has an obvious 'kink' (constant speed sweep that suddenly stops). The rate-limit step creates a velocity discontinuity that the rack may read as an acceleration impulse.

3B.1 -- Switch to the theory-B branch:

```
git fetch origin
git checkout theory-B-pure-filter
```

3B.2 -- Same launch sequence. Try a 0 -> 30 ramp. Should look like a natural ease-in: slow start, fast middle, slow finish, no hard stop.

Tunable: TARGET_FILTER_TAU_S near the top:

```
TARGET_FILTER_TAU_S = 0.40 # was 0.15 on main
```

Larger tau = slower, smoother. Peak velocity for a step input of A is A/tau. At tau=0.40, 30 deg step has peak v = 75 deg/s. If you still see HIGH_ANGLE_RATE_REQ, raise tau to 0.60 or 0.80.

FIX C -- KEEP COMMANDED ANGLES BELOW THE RACK'S STANDSTILL LIMIT

Use this when HIGH_ANGLE_REQ (NOT _RATE_) appears, especially at large angles like +/- 90.

The Tesla EPAS rack scales its maximum allowed steering angle by speed. At 0 km/h the limit is approximately +/- 60 degrees, not +/- 500 (which is the BogGyver-published max for moving vehicles). Trying to command 90 degrees at standstill will trip HIGH_ANGLE_REQ every time.

3C.1 -- During bench testing on jack stands, keep angle commands to **+/- 60 max**.
3C.2 -- The hard angle clamp in the script is fine at +/- 90, you just shouldn't drive it that high without movement.
3C.3 -- For real-world testing, drive slowly (5-15 mph) in an empty lot. The rack will accept much wider angles once it sees vehicle motion via 0x155 ESP_B.

FIX D -- BENCH MODE FOR NO-WHEELS-ON-GROUND TESTING

Use only when the rack is out of the car or you accept the risk of contention with the stock ESP module. See FLASH_AND_TROUBLESHOOTING.pdf section MIN_SPEED for details.

3D.1 -- In tesla_steering_test.py, change BENCH_MODE = False to True near the top.
3D.2 -- Save and restart. The script now sends a fake 0x155 with vehicleSpeed = 30 km/h.
3D.3 -- The rack believes the car is moving and unlocks its full torque curve.

Note: in the car with stock ESP also broadcasting 0x155, our message contends with theirs. Ground-loaded testing with BENCH_MODE = True was unreliable in Charlie's 03 May test (rack hardened but did not actively steer). Recommended only for bench setups with the rack out of the car.

FIX E -- BUS ELECTRICAL ISSUES

Use this when the Bus Errors counter in the GUI grows during testing. Random twitching, mixed error codes, RX counter stalling are all signs.

3E.1 -- **Confirm 60 ohm termination** with a multimeter (car off): probe between OBD-II pins 1 and 9. Should read ~60 ohm. If 120 ohm, you are on a stub. If open, pins not populated.
3E.2 -- **Twist the H/L wires** if they aren't already. Untwisted parallel pairs pick up noise. About 3 turns per inch is fine.
3E.3 -- **Check ground**. SYS TEC pin 3 should connect to OBD-II pin 4 or pin 5 (chassis ground). Without a common ground reference, signaling is unreliable.
3E.4 -- **Try a different USB port** on the laptop. Some USB hubs / front-panel ports have noisier 5V which the SYS TEC sometimes is sensitive to.
3E.5 -- **Inspect connectors**. A loose pin in the OBD-II port or a back-probed wire that shifted will cause intermittent contact and bus errors.

DEEPER DIAGNOSTICS WHEN NOTHING WORKS

If A, B, C, D, and E all failed and you still see flicker, do this:

1. **Capture a 30-second CAN log** using can_sniffer.py. Filter to 0x488, 0x370, 0x101, 0x214, 0x155. Send the capture to Derek along with what GUI showed.
2. **Check 0x101 contention**. The stock GTW broadcasts 0x101 at ~10 Hz with controlType=0. After the firmware patch the rack ignores the value, but if our code somehow ALSO broadcasts 0x101 (IN_CAR_MODE accidentally False), they collide. Confirm IN_CAR_MODE = True near top of tesla_steering_test.py.
3. **Power cycle the rack**. Pull the EPAS high-current fuse from the frunk fuse box for 60 seconds. Reinstall. This forces a clean rack boot in case it got into a weird internal state.
4. **Check car body voltage**. With ignition on, multimeter at 12V battery: should read >13 V (alternator charging) or >12.4 V (engine off). Below that and the rack power is marginal.

KNOWN-GOOD CONFIGURATION CHECKLIST

Before reporting failure, confirm all of these. This is the configuration Charlie's 03 May session worked at (intermittently) and that Derek validated against the BogGyver source:

Check	Expected
EPAS firmware patched (gregjhogan / BogGyver)	Yes -- already done by Derek
Wiring: SYS TEC pin 7 to OBD-II pin 1 (CAN H)	Connected, twisted with pin 9
Wiring: SYS TEC pin 2 to OBD-II pin 9 (CAN L)	Connected
Wiring: SYS TEC pin 3 to OBD-II pin 4 or 5 (GND)	Connected
Termination across pins 1-9 (car off)	~60 ohm
CAN bitrate	500 kbps (set in script)
0x488 send rate	50 Hz
0x488 controlType	1 when engaged, 0 when not
0x488 counter	Increments 0..15 each frame
0x488 checksum	(0x88 + 0x04 + b0 + b1 + b2) and 0xFF
0x101 GTW_epasControl	Sent by stock GTW at ~10 Hz; we do NOT send it (IN_CAR_MODE = True)
0x214 EPB_epasControl	Sent by stock EPB at ~10 Hz; we do NOT send it
0x370 RX rate	~100 frames/sec from rack
Front wheels	OFF the ground (jack stands or tie rods disconnected)
Tesla state	Drive Ready (key in, brake pressed)
Driver torque on wheel	None during engage, otherwise HANDS_ON fault
Commanded angle	0 to +/- 60 deg at standstill

REPORTING TEMPLATE (copy into a text and send to Derek)

Branch: [main / theory-A / theory-B]
Date and time: [...]
Flicker pattern observed: [pattern letter from page 2]
Most common error code in event log: [...]
Bus Errors counter rising: [Y/N]
Measured-vs-commanded divergence: [degrees]
Wheels on ground or jacked: [...]
Tried fixes: [A / B / C / D / E]
Best result so far: [...]
Last 5 lines of GUI event log: [...]